

**Universidade Federal de Lavras
CARLOS HENRIQUE PEREIRA**

ARQUITETURA DE SOFTWARE E ESTRUTURA DE CLASSES: B3 PREDICTIVE BOT

**LAVRAS - MG
2026**

CARLOS HENRIQUE PEREIRA

ARQUITETURA DE SOFTWARE E ESTRUTURA DE CLASSES: B3 PREDICTIVE BOT

Trabalho técnico apresentado como documentação de arquitetura.

**LAVRAS - MG
2026**

Arquitetura de Software e Estrutura de Classes: B3 Predictive Bot

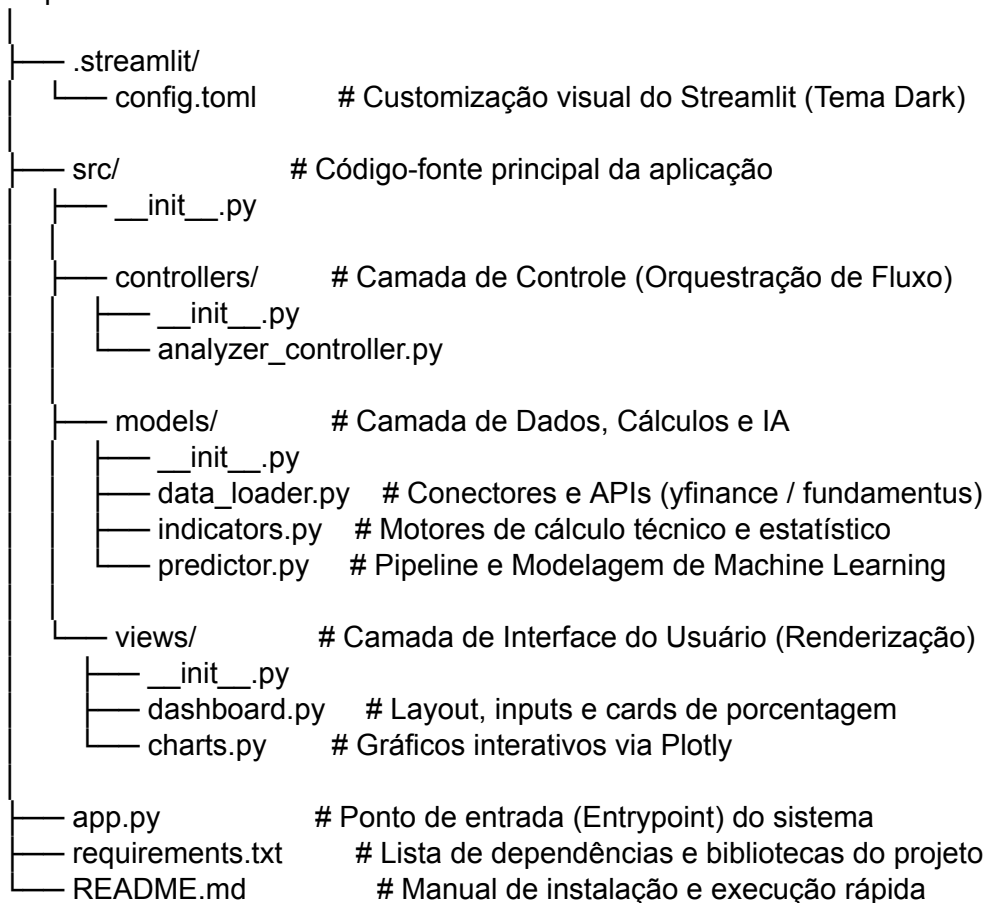
Autor: Inteligência Artificial & Desenvolvimento

Padrão de Arquitetura: Model-View-Controller (MVC) Adaptado

1. Estrutura de Diretórios do Projeto

Para garantir a modularidade, facilidade de manutenção e escalabilidade do robô, o código será organizado na seguinte árvore de arquivos:

b3-predictive-bot/



2. Mapeamento de Classes e Responsabilidades

O sistema utilizará os conceitos de Programação Orientada a Objetos (POO) nas camadas de

dados e controle para encapsular regras de negócio de forma isolada.

Módulo / Classe	Responsabilidade Principal	Principais Métodos
DataLoader (src/models/data_loader.py)	Abstrair o consumo de APIs externas. É a única classe que sabe como se comunicar com o Yahoo Finance e o ecossistema de dados.	• get_ticker_history(ticker, interval, period) • get_financial_metrics(ticker)
IndicatorCalculator (src/models/indicators.py)	Processar os DataFrames brutos e injetar os cálculos matemáticos das 4 camadas analíticas necessárias.	• apply_technical_indicators(df) • apply_quantitative_metrics(df) • detect_candlestick_patterns(df)
MarketPredictor (src/models/predictor.py)	Isolar o ciclo de vida da Inteligência Artificial. Define o alvo de compra, faz o split de treino/teste e calcula as probabilidades.	• prepare_dataset(df, target_horizon) • train_model(X, y) • predict_current_state(latest_data)
AnalyzerController (src/controllers/analyzer_controller.py)	Funciona como o maestro do sistema. Recebe a solicitação da View, aciona os Models na ordem correta e retorna a resposta mastigada.	• run_full_analysis(ticker)

3. Fluxo de Execução da Aplicação

O desacoplamento estrutural garante que as camadas interajam seguindo um fluxo estrito e unidirecional para evitar dependências circulares:

1. O usuário digita o ticker na interface (views/dashboard.py) e clica em analisar.
2. A View chama o método run_full_analysis da classe AnalyzerController.
3. O Controller solicita à DataLoader os dados de mercado e os dados fundamentalistas.
4. Os dados retornados são entregues para a IndicatorCalculator estruturar os indicadores técnicos e matemáticos.
5. O DataFrame enriquecido é passado para a MarketPredictor, que cria a matriz de treino do Machine Learning, treina o modelo sob demanda e realiza a predição para o cenário atual.
6. O Controller junta o veredito percentual da IA com as métricas fundamentalistas e devolve para as Views (dashboard.py e charts.py) renderizarem os resultados finais na tela do investidor.